

Universidad Nacional de San Antonio Abad del Cusco

Departamento Académico de Ingeniería Informática y de Sistemas

Desarrollo de Software II

PRÁCTICA 02

Tipos de Datos: Map, List, Set y más

Lenguaje de Programación Dart

1. Introducción a Dart

Dart es un lenguaje de programación desarrollado por Google, lanzado en 2011 y diseñado para el desarrollo de aplicaciones del lado del cliente. Es el lenguaje base del popular framework Flutter, lo que lo ha convertido en una herramienta esencial para el desarrollo de aplicaciones móviles, web y de escritorio multiplataforma.

Dart es un lenguaje fuertemente tipado, orientado a objetos, con soporte para programación asíncrona, colecciones genéricas y null safety. Su sintaxis es familiar para desarrolladores que vienen de Java, JavaScript o C#.

1.1 Características principales

- Tipado estático con inferencia de tipos
- Null safety integrado desde Dart 2.12
- Compilación JIT (Just-In-Time) y AOT (Ahead-Of-Time)
- Soporte completo para programación orientada a objetos
- Colecciones potentes: List, Set, Map
- Programación asíncrona con async/await y Futures
- Interoperabilidad con JavaScript

¿Por qué aprender Dart?

Dart es el lenguaje oficial de Flutter, el framework de Google para crear apps nativas en iOS, Android, Web y Desktop con un solo código base. Conocer Dart es la puerta de entrada al ecosistema Flutter.

2. Tipos de Datos Primitivos

Dart cuenta con varios tipos de datos primitivos incorporados que forman la base del lenguaje. A continuación se presenta una tabla resumen y ejemplos prácticos de cada uno.

Tipo	Ejemplo	Descripción
int	int edad = 25;	Números enteros (positivos, negativos o cero)
double	double pi = 3.14;	Números de punto flotante de doble precisión
String	String nombre = "Dart";	Cadenas de texto inmutables
bool	bool activo = true;	Valores lógicos: true o false
num	num valor = 42;	Superclase de int y double
dynamic	dynamic x = 'hola';	Tipo dinámico, puede cambiar en ejecución
var	var n = 10;	Inferencia de tipo en tiempo de compilación

2.1 Ejemplos de tipos primitivos

```
// Tipos numéricos
int entero = 100;
double decimal = 3.14159;
num numero = 42;          // puede ser int o double

// Cadenas de texto
String saludo = "Hola, Dart!";
String interpolado = 'El valor es: ${entero}';
String multilinea = '''
    Esta es una cadena
    en múltiples líneas
''';

// Booleanos
bool esDart = true;
bool esPython = false;

// Null safety
int? edadNula = null; // el ? indica que puede ser null
String nombre = 'Ana'; // nunca puede ser null
```

3. Tipo de Dato: List (Listas)

Una List en Dart es una colección ordenada de elementos. Equivale a los arrays en otros lenguajes, pero con funcionalidades mucho más ricas. Las listas pueden ser de longitud fija o dinámica, y pueden ser genéricas (tipadas) o no tipadas.

3.1 Creación de Listas

```
// Lista literal
List<int> numeros = [1, 2, 3, 4, 5];

// Lista vacía con tipo genérico
List<String> nombres = [];

// Con var (inferencia de tipo)
var frutas = ['manzana', 'pera', 'uva'];

// Lista de longitud fija
List<int> fija = List.filled(3, 0); // [0, 0, 0]

// Lista con generador
List<int> cuadrados = List.generate(5, (i) => i * i);
// Resultado: [0, 1, 4, 9, 16]
```

3.2 Operaciones principales sobre List

Método / Propiedad	Descripción y ejemplo
.add(elem)	Agrega un elemento al final: lista.add(6);
.addAll(lista)	Agrega todos los elementos de otra lista
.insert(i, elem)	Inserta en posición i: lista.insert(0, 99);
.remove(elem)	Elimina la primera coincidencia
.removeAt(i)	Elimina el elemento en el índice i
.contains(elem)	Retorna true si existe el elemento
.indexOf(elem)	Retorna el índice del elemento
.length	Número de elementos en la lista
.isEmpty / .isNotEmpty	Verifica si la lista está vacía
.reversed	Retorna un iterable en orden inverso
.sort()	Ordena la lista en su lugar
.map(f)	Transforma cada elemento con función f
.where(f)	Filtra elementos que cumplen condición
.forEach(f)	Itera sobre cada elemento
.reduce(f)	Reduce la lista a un solo valor

3.3 Ejemplo práctico con List

```
void main() {
    List<String> estudiantes = ['Ana', 'Luis', 'María', 'Carlos'];

    // Agregar elemento
    estudiantes.add('Pedro');

    // Recorrer con forEach
    estudiantes.forEach((e) => print('Estudiante: $e'));

    // Filtrar nombres con más de 4 letras
    var largos = estudiantes.where((e) => e.length > 4).toList();
    print(largos); // [María, Carlos, Pedro]

    // Transformar a mayúsculas
    var mayusculas = estudiantes.map((e) => e.toUpperCase()).toList();
    print(mayusculas);

    // Ordenar
    estudiantes.sort();
    print(estudiantes);
}
```

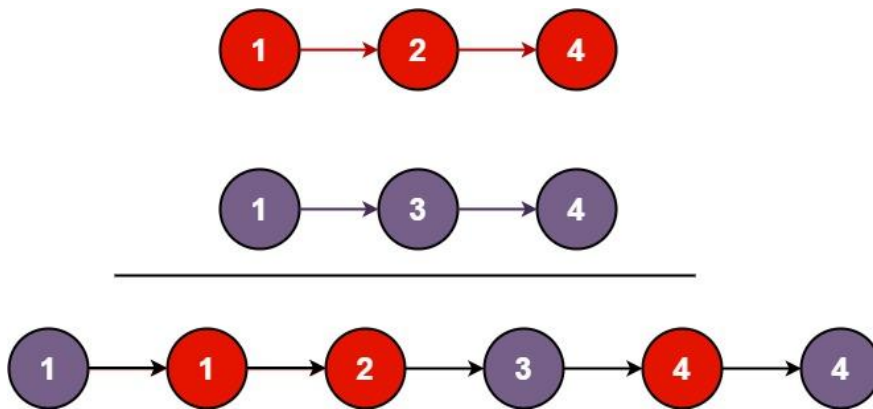
3.3 Ejercicio propuesto con List

Se te proporcionan los nodos iniciales de dos listas enlazadas ordenadas: lista1 y lista2.

Combina las dos listas en una sola lista ordenada. Esta lista se formará uniendo los nodos de las dos primeras listas.

Devuelve el nodo inicial de la lista enlazada resultante.

Ejemplo 1:



Entrada: lista1 = [1,2,4], lista2 = [1,3,4]

Salida: [1,1,2,3,4,4]

Ejemplo 2:

Entrada: lista1 = [], lista2 = []

Salida: []

Ejemplo 3:

Entrada: lista1 = [], lista2 = [0]

Salida: [0]

Restricciones:

El número de nodos en ambas listas está en el rango [0, 50].

- $-100 \leq \text{Node.val} \leq 100$
- Tanto la lista1 como la lista2 están ordenadas en orden no decreciente.

4. Tipo de Dato: Map (Mapas)

Un Map es una colección de pares clave-valor. Cada clave es única dentro del mapa y se usa para acceder a su valor asociado. Es equivalente a los diccionarios en Python o los objetos en JavaScript. En Dart los Maps son genéricos: `Map<TipoClave, TipoValor>`.

4.1 Creación de Maps

```
// Map literal
Map<String, int> edades = {
  'Ana': 25,
  'Luis': 30,
  'María': 22,
};

// Con var (inferencia)
var capitales = {
  'Perú': 'Lima',
  'Bolivia': 'Sucre',
  'Chile': 'Santiago',
};

// Map vacío
Map<String, dynamic> datos = {};

// Constructor
var mapa = Map<String, int>();
```

4.2 Operaciones principales sobre Map

Método / Propiedad	Descripción y ejemplo
map[clave]	Accede al valor: edades['Ana'] → 25
map[clave] = val	Asigna o actualiza: edades['Ana'] = 26;
.putIfAbsent(k, f)	Agrega solo si la clave no existe
.remove(clave)	Elimina la entrada con esa clave
.containsKey(k)	true si existe la clave
.containsValue(v)	true si existe ese valor en alguna entrada
.keys	Iterable con todas las claves
.values	Iterable con todos los valores
.entries	Iterable de MapEntry<K, V>
.length	Número de pares clave-valor
.isEmpty / .isNotEmpty	Verifica si el mapa está vacío
.forEach((k,v)=>{})	Itera sobre cada par clave-valor
.map((k,v)=>...)	Transforma entradas, retorna nuevo Map
.update(k, f)	Actualiza valor existente con función
.addAll(otroMap)	Fusiona otro mapa al actual

4.3 Ejemplo práctico con Map

```
void main() {
    // Inventario de una tienda
    Map<String, double> precios = {
        'Manzana': 2.50,
        'Pera': 3.00,
        'Uva': 5.50,
    };

    // Acceder a un valor
    print(precios['Manzana']); // 2.5

    // Agregar nuevo producto
    precios['Mango'] = 4.75;

    // Actualizar precio
    precios.update('Pera', (v) => v * 1.1);

    // Iterar sobre el mapa
    precios.forEach((producto, precio) {
        print('$producto: S/. ${precio.toStringAsFixed(2)}');
    });
}
```

```
});

// Filtrar productos baratos (precio < 4)
var baratos = Map.fromEntries(
  precios.entries.where((e) => e.value < 4),
);
print('Productos económicos: $baratos');
}
```

4.4 Maps anidados

```
// Map dentro de Map (estructura JSON-like)
Map<String, Map<String, dynamic>> estudiantes = {
  'A001': {
    'nombre': 'Ana García',
    'edad': 22,
    'nota': 18.5,
  },
  'A002': {
    'nombre': 'Luis Pérez',
    'edad': 24,
    'nota': 16.0,
  },
};

// Acceder a datos anidados
print(estudiantes['A001']['nombre']); // Ana García
print(estudiantes['A002']['nota']);   // 16.0
```

4.5 Ejercicio propuesto con Maps

Dados dos arreglos de enteros, `nums1` y `nums2`, devuelve un arreglo con sus elementos en intersección. Cada elemento del resultado debe aparecer tantas veces como aparece en ambos arreglos, y puedes devolver el resultado en cualquier orden.

Ejemplo 1:

Entrada: `nums1 = [1,2,2,1]`, `nums2 = [2,2]`

Salida: `[2,2]`

Ejemplo 2:

Entrada: `nums1 = [4,9,5]`, `nums2 = [9,4,9,8,4]`

Salida: `[4,9]`

Explicación: `[9,4]` también es válido.

Restricciones:

- `1 <= nums1.length, nums2.length <= 1000`

- $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 1000$

5. Tipo de Dato: Set (Conjuntos)

Un Set es una colección no ordenada de elementos únicos. No permite duplicados. Es ideal para operaciones de conjuntos matemáticos como unión, intersección y diferencia. Internamente Dart implementa Set como un LinkedHashMap, que mantiene el orden de inserción.

5.1 Creación de Sets

```
// Set literal (notar las llaves {})
Set<int> numeros = {1, 2, 3, 4, 5};

// Conjunto de Strings
var colores = {'rojo', 'verde', 'azul'};

// Set vacío (;usar Set<tipo>(), no {} que crea Map!)
Set<String> vacio = {};
// o bien:
var vacio2 = <String>{};

// Elimina duplicados automáticamente
Set<int> sinDups = {1, 2, 2, 3, 3, 3};
print(sinDups); // {1, 2, 3}
```

5.2 Operaciones de conjuntos

```
Set<int> A = {1, 2, 3, 4, 5};
Set<int> B = {4, 5, 6, 7, 8};

// Unión
print(A.union(B)); // {1, 2, 3, 4, 5, 6, 7, 8}

// Intersección
print(A.intersection(B)); // {4, 5}

// Diferencia
print(A.difference(B)); // {1, 2, 3}

// Subconjunto
Set<int> C = {1, 2};
print(C.isSubsetOf(A)); // true

// Agregar elemento (ignorado si ya existe)
A.add(6);
A.addAll({7, 8});
```

5.3 Ejercicio propuesto con conjuntos

Se te proporcionan dos arreglos de enteros, `frutas` y `cestas`, cada uno de longitud n , donde `frutas[i]` representa la cantidad del i -ésimo tipo de fruta y `cestas[j]` representa la capacidad de la j -ésima cesta.

De izquierda a derecha, coloca las frutas según estas reglas:

Cada tipo de fruta debe colocarse en la cesta disponible más a la izquierda con una capacidad mayor o igual a la cantidad de ese tipo de fruta.

Cada cesta solo puede contener un tipo de fruta.

Si un tipo de fruta no se puede colocar en ninguna cesta, queda sin colocar.

Devuelve el número de tipos de fruta que quedan sin colocar después de realizar todas las asignaciones posibles.

Ejemplo 1:

Entrada: frutas = [4,2,5], cestas = [3,5,4]

Salida: 1

Explicación:

4 frutas[0] se colocan en 5 cestas[1].
2 frutas[1] se colocan en 3 cestas[0].
5 frutas[2] no se pueden colocar en 4 cestas[2].
Como queda un tipo de fruta sin colocar, devolvemos 1.

Ejemplo 2:

Entrada: frutas = [3,6,1], cestas = [6,4,7]

Salida: 0

Explicación:

3 frutas[0] se colocan en 6 cestas[0].
6 frutas[1] no se pueden colocar en 4 cestas[1] (capacidad insuficiente), pero se pueden colocar en la siguiente cesta disponible, 7 cestas.
1 fruta[2] Se coloca en la cesta[1] = 4.
Como todas las frutas se colocan correctamente, devolvemos 0.

Restricciones:

- $n == \text{longitud_frutas} == \text{longitud_cestas}$

- $1 \leq n \leq 100$
- $1 \leq \text{frutas}[i], \text{cestas}[i] \leq 1000$

6. Otros Tipos de Datos en Dart

6.1 Runes y Symbols

```
// Runes: puntos de código Unicode
Runes corazon = Runes('\u2764');
print(String.fromCharCode(corazon)); // ♥

// Symbols: identificadores en Dart
Symbol sym = #nombreVariable;
print(sym); // Symbol('nombreVariable')
```

6.2 Records (Dart 3.0+)

Los Records son tipos de datos compuestos, inmutables y de tamaño fijo, introducidos en Dart 3.0. Permiten agrupar múltiples valores sin necesidad de crear una clase.

```
// Record con campos posicionales
var punto = (10.0, 20.0);
print(punto.$1); // 10.0
print(punto.$2); // 20.0

// Record con campos nombrados
var persona = (nombre: 'Ana', edad: 25);
print(persona.nombre); // Ana

// Función que retorna múltiples valores
(String, int) obtenerInfo() {
  return ('Dart', 3);
}
var (lenguaje, version) = obtenerInfo();
```

6.3 Enumeraciones (Enum)

```
// Enum básico
enum Direccion { norte, sur, este, oeste }

// Enum con propiedades (Dart 2.17+)
enum Planeta {
  tierra(9.8),
  marte(3.7),
  jupiter(24.8);

  final double gravedad;
  const Planeta(this.gravedad);
}
```

```
void main() {
  var dir = Direccion.norte;
  print(dir.name); // norte

  print(Planeta.marte.gravedad); // 3.7
}
```

6.4 Null Safety

Desde Dart 2.12, el lenguaje incluye Null Safety: el sistema de tipos distingue entre variables que pueden ser null y las que no. Esto previene los errores de NullPointerException en tiempo de compilación.

```
// Variable NO nullable (nunca puede ser null)
String nombre = 'Dart';

// Variable NULLABLE (puede ser null)
String? apellido = null;

// Operador de aserción de null (!)
print(apellido!.length); // Error en runtime si es null

// Operador de acceso seguro (?.)
print(apellido?.toUpperCase()); // null si es null, no error

// Operador de coalescencia null (??)
String resultado = apellido ?? 'Sin apellido';

// Asignación condicional null (??=)
apellido ??= 'García'; // asigna solo si era null
```

7. Rúbrica de Evaluación

La siguiente rúbrica permite evaluar el dominio de los tipos de datos en Dart en cuatro niveles de desempeño.

Criterio	Excelente (4)	Bueno (3)	Regular (2)	Insuficiente (1)
Comprensión de tipos primitivos	Identifica y usa correctamente todos los tipos: int, double, String, bool, dynamic, var con null safety.	Usa la mayoría de los tipos correctamente con errores menores.	Usa tipos básicos pero confunde dynamic, var o ignora null safety.	No distingue los tipos o comete errores frecuentes.
Uso de List	Crea, accede, filtra, transforma y ordena listas usando métodos avanzados	Usa add, remove, forEach correctamente con algún método funcional.	Crea listas y usa acceso por índice, pero no usa métodos funcionales.	No puede crear ni manipular listas correctamente.

	(map, where, reduce).			
Uso de Map	Crea Maps genéricos, usa forEach, entries, update, putIfAbsent y Maps anidados.	Crea Maps, accede por clave y los recorre con forEach.	Crea Maps literales y accede a valores, con dificultades para iterar.	Confunde Map con otros tipos o no puede acceder a sus valores.
Uso de Set	Aplica unión, intersección, diferencia y evita duplicados en contextos reales.	Crea Sets y usa operaciones básicas de conjuntos.	Crea Sets pero no aprovecha su naturaleza de elementos únicos.	No distingue Set de List o no puede crear uno correctamente.
Calidad del código	Código limpio, bien nombrado, sin repetición, con tipos explícitos y comentarios.	Código organizado con buenos nombres de variables.	Código funcional pero difícil de leer o con nombres poco claros.	Código desorganizado o con errores de estilo graves.

Puntaje total máximo: 20 puntos | Escala: Excelente (18–20) | Bueno (16–17) | Regular (10–15) | Insuficiente (< 10)

8. Referencias Bibliográficas

Dart Team. (2024). *The Dart language tour*. Google LLC. <https://dart.dev/language>

Dart Team. (2024). *Core libraries — dart:core*. Google LLC. <https://api.dart.dev/stable/dart-core/dart-core-library.html>

Biessek, A. (2023). *Flutter and Dart Cookbook*. O'Reilly Media.

Akopkokhyants, S. (2023). *Dart: Up and Running* (2.ª ed.). O'Reilly Media.

Google LLC. (2023). *Dart null safety: Understanding null safety*. <https://dart.dev/null-safety/understanding-null-safety>

Google LLC. (2024). *Records — Dart 3 language features*. <https://dart.dev/language/records>

Windmill, E., & Biessek, A. (2021). *Flutter in Action*. Manning Publications.
